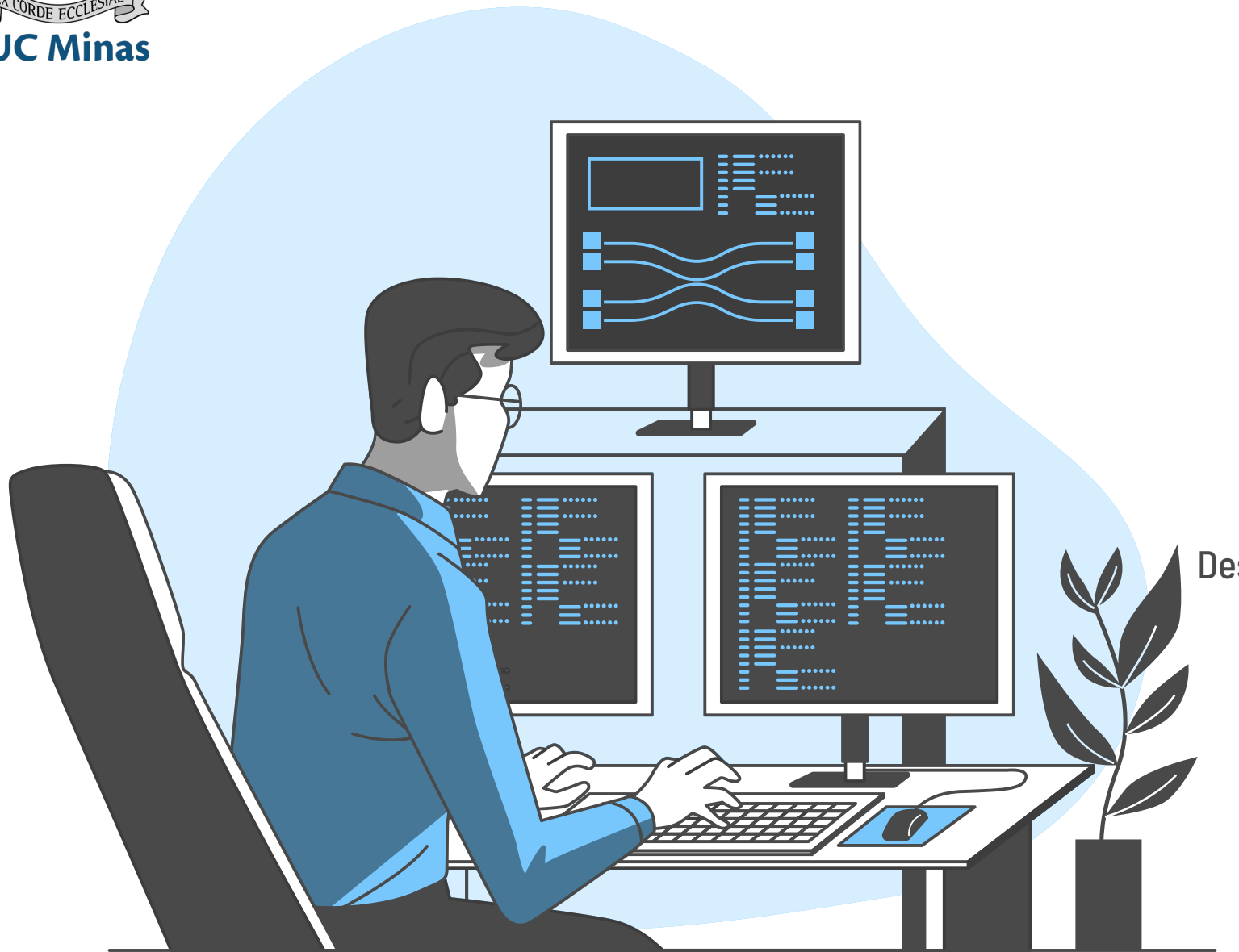
PUC Minas



Aula 08 · Deploy

Produção sem sustos: diagnóstico e checklist

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Aplicar um **método de diagnóstico** camada por camada
2. Saber **onde ler os logs** do Vercel, do Render, do Atlas e do Supabase
3. Reconhecer os erros mais comuns de produção
4. Proteger **segredos** e o que fazer se um vazar
5. Usar o **checklist de entrega**

Antes de começar

- Aulas 01 a 07 desta pasta

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O método: de fora para dentro

02 Onde estão os logs

03 Os erros mais comuns, de ponta a ponta

04 Segredos

05 Checklist de entrega

06 Aprofundando: árvore de decisão para incidentes

01

O método: de fora para dentro

O método: de fora para dentro

Quando “não funciona”, resista à vontade de mudar código. Teste cada camada, da mais externa para a mais interna, e pare na primeira que falhar:

1. **O site abre?** Acesse a URL do Vercel. Tela branca → Console do DevTools (erro de JavaScript) ou log de build do Vercel.
2. **O back responde sozinho?** Abra `https://seu-back.onrender.com/api/status` no navegador. Não abre → logs do Render.
3. **O banco está acessível?** O status mostra o banco `offline`, ou o log tem *timeout* → Network Access do Atlas / pooler do Supabase / projeto pausado.
4. **O front chama a URL certa?** Aba Network: o domínio é o do Render? Se for `localhost`, falta `VITE_API_URL` + Redeploy.
5. **O navegador permite?** Erro de CORS no Console → `CORS_ORIGINS` no Render.

O método: de fora para dentro (cont.)

6

A resposta é um erro da regra de negócio? Leia o `mensagem` do JSON de erro na aba Network.

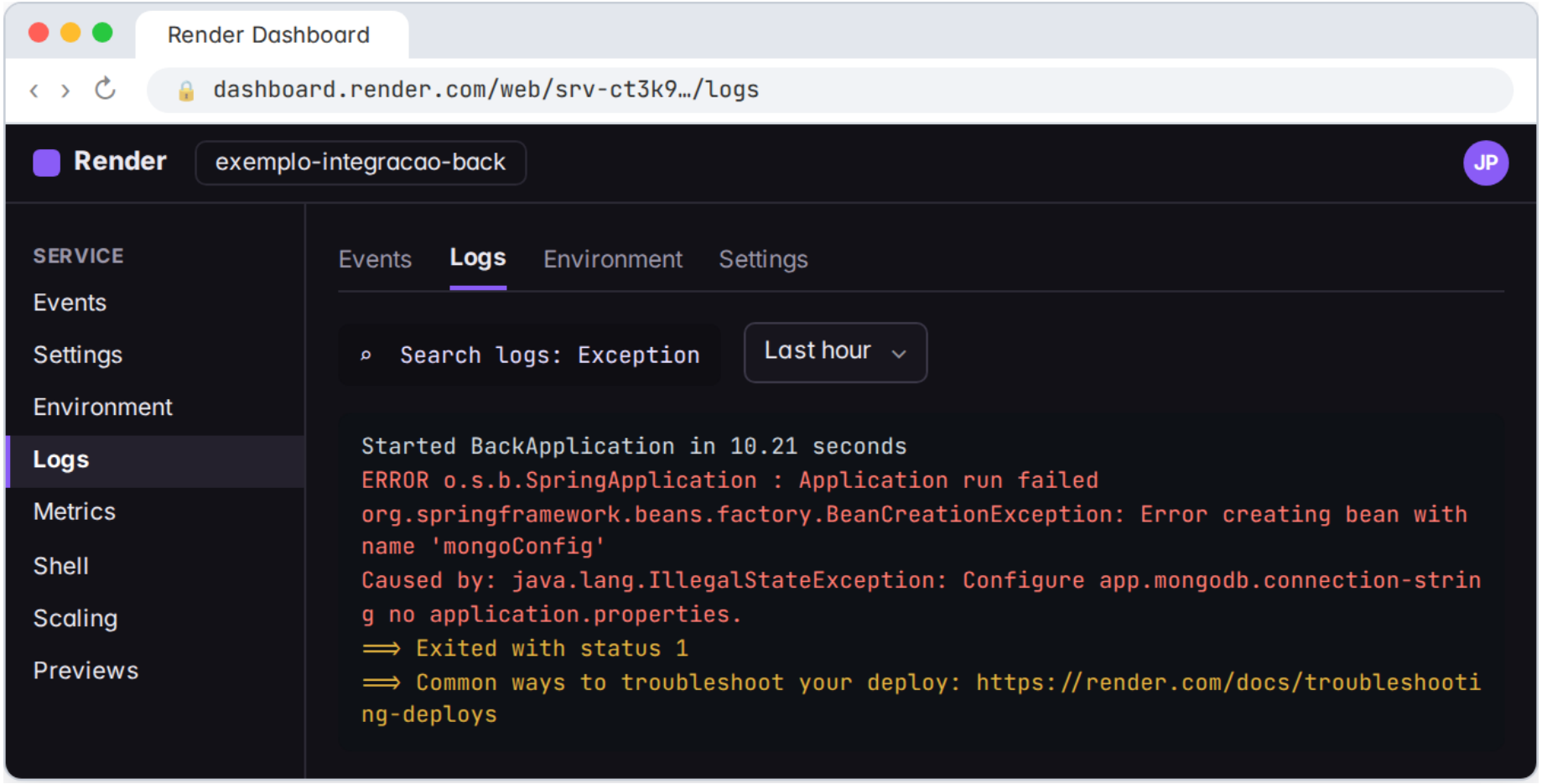
02

Onde estão os logs

Onde estão os logs

Serviço	Onde	O que procurar
Vercel (build)	Project → Deployments → clique no deploy → *Building*	`npm install`/`vite build` falhando, import com maiúscula errada
Vercel (navegador)	DevTools do seu navegador	Console (JS, CORS) e Network (URLs, status)
Render	Service → **Logs** e **Events**	Erro do Docker, exceção do Spring na inicialização, `OutOfMemory`
MongoDB Atlas	Cluster → **Metrics** / Project → **Activity Feed**	Conexões, cluster pausado
Supabase	Project → **Logs** (Postgres, API)	Falha de autenticação, conexões recusadas, RLS

Onde estão os logs



Leia o log de baixo para cima e procure o primeiro **Caused by**: aqui, a variável `MONGODB_URI` não foi cadastrada.

Onde estão os logs

Dica

As mensagens de erro claras como esta não são sorte: o `MongoConfig` do `Exemplo_Integracao` lança `IllegalStateException` com uma frase explicando o que configurar. Escreva mensagens assim no seu código; elas economizam horas em produção.

03

Os erros mais comuns, de ponta a ponta

Os erros mais comuns, de ponta a ponta

Sintoma	Camada	Causa mais provável
Build do Vercel falha com `Could not resolve`	Front/build	Import com maiúsculas diferentes do arquivo
Site em branco, Console com `Uncaught ReferenceError`	Front/JS	Erro de código; rode `npm run build && npm run preview` local
F5 em rota dá 404	Front/Vercel	Falta `vercel.json`
Network mostra chamadas para `localhost:8080`	Front/variáveis	`VITE_API_URL` ausente ou sem Redeploy
Erro de CORS	Back/config	`CORS_ORIGINS` sem a URL exata do Vercel
Primeira chamada demora ~50 s	Back/Render	*Cold start* do plano Free

Os erros mais comuns, de ponta a ponta (cont.)

Sintoma	Camada	Causa mais provável
Render: `Exited with status 1`	Back/inicialização	Variável faltando, banco inacessível, porta
`MongoTimeoutException`	Banco	Atlas sem `0.0.0.0/0` ou cluster pausado
`UnknownHostException db.xxx.supabase.co`	Banco	*Direct connection* (IPv6) em vez do pooler
`401` após reiniciar o back	Back/JWT	Chave gerada em memória; use `JWT_SECRET` fixo
Funciona no Docker local, falha no Render	Back/deploy	Root Directory, variável não cadastrada ou memória (512 MB)

04

Segredos

Segredos

Vazou um segredo?

Se uma senha ou chave foi parar no GitHub, considere-a vazada, mesmo que você apague o arquivo no commit seguinte (ela continua no histórico). Faça na hora:

- **Troque** a credencial: nova senha do usuário no Atlas (*Database Access* → *Edit* → *Edit Password*), *Reset database password* no Supabase, nova *secret key* no Supabase, novo `JWT_SECRET`.
- **Atualize** a variável no Render/Vercel e faça o deploy.
- **Remova** o valor do código e coloque `${VARIAVEL}` no lugar.

O GitHub costuma enviar um e-mail de *secret scanning* quando detecta chaves conhecidas; não ignore.

Segredos

Segredo	Onde fica	Nunca em
Senha do MongoDB / Supabase	Variáveis do **Render**	`application.properties` versionado, variáveis `VITE_`
`JWT_SECRET`	Variáveis do **Render**	Código Java, front
Secret key do Supabase	Variáveis do **Render** (se o back precisar)	Front-end
URL da API, *publishable key*	Variáveis do **Vercel**	(são públicas, mas mantêm o código limpo)

05

Checklist de entrega

Checklist de entrega

Use esta lista antes de entregar um trabalho ou apresentar:

✓	Item
☐	Repositório público com `Back/`, `Front/` e **README** (descrição, tecnologias, URLs de produção, como rodar, variáveis)
☐	Nenhum `node_modules/`, `target/`, `dist/`, `.env.local` ou senha real versionados
☐	`.env.example` no Front e variáveis documentadas no README do Back
☐	`pom.xml` com `25` e Dockerfile com imagens **temurin-25**
☐	`server.port=${PORT:8080}` e todas as configurações sensíveis como `${VARIAVEL}`
☐	`docker build` e `docker run` do back testados localmente

Checklist de entrega (cont.)

✓	Item
☐	Back no Render, **Live**, com Health Check em `/api/status`
☐	Front no Vercel com Root Directory `Front`, `VITE_API_URL` e `vercel.json`
☐	`CORS_ORIGINS` com a URL exata do Vercel
☐	Atlas com `0.0.0.0/0` **ou** Supabase com *Session pooler*
☐	CRUD completo testado **pelo site do Vercel**, com DevTools sem erros
☐	Back acordado e Supabase ativo **minutos antes** da apresentação

Checklist de entrega

Observação

Quer ir além do plano gratuito? Os mesmos conceitos valem para outros serviços: **Fly.io** (há um projeto `Fly_io_View_RestAPI` no repositório), **Railway**, **Google Cloud Run** e **AWS App Runner** também rodam containers a partir de um Dockerfile. **Netlify** e **Cloudflare Pages** publicam o front como o Vercel.

06

Aprofundando: árvore de decisão para incidentes

Aprofundando: árvore de decisão para incidentes

Pergunta	Se NÃO	Se SIM
O site do Vercel abre?	Veja o log de build do Vercel e o Console	Próxima pergunta
`/api/status` do Render responde?	Logs do Render (inicialização, memória, porta)	Próxima pergunta
O status mostra o banco online?	Network Access do Atlas / pooler e pausa do Supabase	Próxima pergunta
A aba Network chama o domínio do Render?	`VITE_API_URL` + Redeploy no Vercel	Próxima pergunta
Há erro de CORS no Console?	Leia o `mensagem` do JSON de erro	`CORS_ORIGINS` no Render

Pós-incidente (post-mortem) em 4 perguntas

O que aconteceu?

Sintoma visto pelo usuário e horário.

Por quê?

Causa raiz, não só o sintoma (ex.: variável removida no Render).

Como resolvemos?

Os passos exatos, para repetir se acontecer de novo.

Como evitar?

Teste, checklist ou alerta novo. Sem procurar culpados.

Aprofundando: árvore de decisão para incidentes

Seção de deploy do README

```
## Produção
| Parte | URL |
| --- | --- |
| Front | https://exemplo-integracao-front.vercel.app |
| API | https://exemplo-integracao-back.onrender.com |
| Status | https://exemplo-integracao-back.onrender.com/api/status |

## Variáveis
| Serviço | Variável | Descrição |
| --- | --- | --- |
| Render | MONGODB_URI | connection string do Atlas (segredo) |
| Render | CORS_ORIGINS | URLs do front separadas por vírgula |
| Vercel | VITE_API_URL | URL do back, sem barra final |
```

Corrigir em produção x reproduzir localmente

Mexer direto no painel

- Rápido para um ajuste de variável
- Fácil esquecer de documentar
- Arriscado para código

vs

Reproduzir localmente

- Mesmo Docker e variáveis
- Correção testada antes do push
- Caminho padrão para bugs de código

Aprofundando: árvore de decisão para incidentes

Dica

Guarde em um documento os **erros que você já resolveu**, com a mensagem exata e a solução. Os mesmos problemas (CORS, IPv6 do Supabase, Network Access do Atlas, Redeploy esquecido) voltam em todo projeto.

07

Quiz de revisão

Pergunta

Qual o primeiro teste quando “não funciona”?

A Mudar o código

B Ver se o site abre e se o back responde sozinho

C Reinstalar o Docker

D Trocar de banco

Resposta

Qual o primeiro teste quando “não funciona”?

A Mudar o código

✓ Ver se o site abre e se o back responde sozinho

C Reinstalar o Docker

D Trocar de banco

Por quê: Diagnostique de fora para dentro.

Pergunta

No log do Render, o que procurar?

A. A primeira linha

B. O primeiro Caused by, lendo de baixo para cima

C. O banner do Spring

D. Nada

Resposta

No log do Render, o que procurar?

A. A primeira linha

✓ O primeiro Caused by, lendo de baixo para cima

C. O banner do Spring

D. Nada

Por quê: Ele mostra a causa real.

Pergunta

Um segredo foi para o GitHub. O que fazer primeiro?

A Apagar o commit

B Trocar a credencial no serviço

C Tornar o repositório privado e pronto

D Nada

Resposta

Um segredo foi para o GitHub. O que fazer primeiro?

A Apagar o commit

✓ Trocar a credencial no serviço

C Tornar o repositório privado e pronto

D Nada

Por quê: Ele continua no histórico.

Pergunta

O status mostra o banco offline. Onde olhar?

A vercel.json

B Network Access do Atlas / pooler e pausa do Supabase

C package.json

D CORS

Resposta

O status mostra o banco offline. Onde olhar?

A vercel.json

✓ Network Access do Atlas / pooler e pausa do Supabase

C package.json

D CORS

Por quê: O back está no ar, mas não alcança o banco.

Pergunta

Qual item é parte do checklist de entrega?

A node_modules no Git

B CRUD testado pelo site do Vercel com o DevTools sem erros

C Senha no application.properties

D Front só local

Resposta

Qual item é parte do checklist de entrega?

A node_modules no Git

✓ CRUD testado pelo site do Vercel com o DevTools sem erros

C Senha no application.properties

D Front só local

Por quê: E o back acordado antes de apresentar.

Fechamento

Resumo da aula

- ✓ Diagnostique de fora para dentro: site → back sozinho → banco → URL chamada → CORS → mensagem de erro.
- ✓ Logs: Vercel (build), DevTools (navegador), Render (Logs/Events), Atlas e Supabase (Logs/Metrics).
- ✓ No log do Render, procure o primeiro **Caused by**.
- ✓ Segredo vazado: troque, atualize a variável, remova do código.
- ✓ Siga o checklist antes de entregar.

Exercícios

1. Provoque três falhas diferentes em produção (CORS, variável do banco, `VITE_API_URL`) e documente o sintoma e o log de cada uma.

2. Escreva o README final do seu projeto seguindo o primeiro item do checklist.

3. Simule um vazamento: troque a senha do usuário do Atlas e atualize o Render sem derrubar o site por mais de 2 minutos.

4. Preencha o checklist de entrega e anexe ao trabalho.

Para ir além

- render.com/docs/troubleshooting-deploys
- vercel.com/docs/deployments/troubleshoot-a-build
- docs.github.com/code-security/secret-scanning

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 08?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web